check_paired.py 사용법 및 코드 설명서

대상 파일: `lab11/src/check_paired.py`

1. 프로그램의 목적

paired-end 시퀀싱에서는 DNA 조각 하나의 양쪽 끝을 읽은 결과가 두 파일(`_1.fastq`, `_2.fastq`)에 나뉘어 저장된다. 두 파일의 **같은 순서(i번째)에 있는 read끼리 짝**이므로, 파일 하나에서 read가 빠지거나 순서가 바뀌면 이후 분석(조립, 매핑)이 잘못된다.

이 프로그램은 두 FASTQ 파일이 서로 짝이 맞는지 다음 두 가지로 확인한다.

1. 두 파일의 **read 개수가 같은지** 확인한다.
2. 같은 순서에 있는 read끼리 **read 이름(ID)이 일치하는지** 확인한다.

두 조건을 모두 만족해야 "짝이 맞는 paired-end FASTQ"로 판정한다.

2. 실행 방법

`lab11` 폴더에서 다음과 같이 실행한다.

```
python3 src/check_paired.py data/SRR1553607_1.fastq data/SRR1553607_2.fastq
```

형식:

```
python3 src/check_paired.py <FASTQ_1> <FASTQ_2>
```

- 첫 번째 인자(`sys.argv[1]`)는 `_1` 파일, 두 번째 인자(`sys.argv[2]`)는 `_2` 파일이다. 순서를 바꿔도 판정 결과는 같고, 출력에서 "파일 1", "파일 2"의 위치만 바뀐다.
- 인자 개수를 검사하는 코드는 없다. 인자를 하나만 주면 `sys.argv[2]`에서 `IndexError: list index out of range`가 난다.
- 두 파일을 모두 메모리에 읽는다. 아래 실행에서는 약 1초가 걸렸다.

3. 입력 파일

FASTQ 형식의 텍스트 파일 두 개이다. read 하나는 **4줄**(헤더, 서열, `+` 줄, 품질 문자열)로 이루어진다. 이 프로그램은 4줄 중 **첫 줄(헤더)만** 사용한다.

파일	내용
data/SRR1553607_1.fastq	paired-end의 첫 번째 read들 (813,780줄)
data/SRR1553607_2.fastq	paired-end의 두 번째 read들 (813,780줄)

이 데이터의 헤더는 다음과 같은 SRA 형식이다. 두 파일에서 헤더 전체가 똑같다.

```
@SRR1553607.1 1 length=101
```

4. read 이름 비교 방법

FASTQ 헤더 형식은 시퀀서와 데이터 출처에 따라 다르다. 짝이 되는 read라도 헤더 **전체**가 항상 같지는 않으므로, 이 프로그램은 헤더에서 **read ID만 뽑아** 비교한다.

1. 첫 번째 공백 앞부분만 read ID로 쓴다.
2. ID 끝에 `/1` 또는 `/2`가 붙어 있으면 떼고 비교한다.

형식	_1 헤더	_2 헤더	비교에 쓰는 ID
SRA (이 데이터)	@SRR1553607.1 1 length=101	@SRR1553607.1 1 length=101	SRR1553607.1
새 일루미나 (Casava 1.8 이후)	@M001:1:2 1:N:0:1	@M001:1:2 2:N:0:1	M001:1:2
예전 일루미나	@HWI:1:1#0/1	@HWI:1:1#0/2	HWI:1:1#0

새 일루미나 형식에서는 공백 뒤의 `1:N:0:1` / `2:N:0:1`이 서로 다르고, 예전 형식에서는 ID 끝의 `/1` / `/2`가 서로

다르다. 위 두 규칙으로 이 차이를 없앤 뒤 비교한다.

5. 출력 결과

결과는 화면에 출력되며 파일로 저장하지 않는다. 실제 실행 결과는 다음과 같다.

```
파일 1: data/SRR1553607_1.fastq
파일 2: data/SRR1553607_2.fastq

[1] read 개수 확인
  파일 1 read 개수: 203445
  파일 2 read 개수: 203445
  결과: 일치

[2] 짝이 되는 read 이름 확인
  비교한 read 쌍: 203445
  이름이 일치한 쌍: 203445
  이름이 다른 쌍: 0
  첫 번째 쌍 예시: SRR1553607.1 1 length=101  ┆  SRR1553607.1 1 length=101
  비교에 쓴 ID: SRR1553607.1  ┆  SRR1553607.1

[최종 결과]
  두 파일은 짝이 맞는 paired-end FASTQ입니다.
```

출력 부분	의미
[1] read 개수 확인	파일별 read 개수와 일치 여부. 다르면 차이 개수를 함께 출력한다.
[2] 짝이 되는 read 이름 확인	비교한 쌍 수, 일치한 쌍 수, 다른 쌍 수. 다른 쌍이 있으면 처음 5개까지 몇 번째 read인지 출력한다.
첫 번째 쌍 예시 / 비교에 쓴 ID	실제 헤더와 그 헤더에서 뽑은 ID. 비교 방법이 올바른지 눈으로 확인하는 용도이다.
[최종 결과]	개수와 이름이 모두 일치하면 짝이 맞는다고, 하나라도 다르면 맞지 않는다고 출력한다.

6. 결과 해석

- 두 파일의 read 개수는 각각 **203,445개**이다. `wc -l`로 센 줄 수 813,780을 4로 나눈 값과 같다.
- 203,445쌍 모두 read ID가 일치했다. 따라서 `SRR1553607_1.fastq`와 `SRR1553607_2.fastq`는 짝이 맞는 paired-end 파일이며, 그대로 paired-end 분석 도구에 넣을 수 있다.

7. 코드 설명

7.1 `read_names(filename)` — 헤더 줄 모으기

```
def read_names(filename):
    names = []
    with open(filename) as f:
        lines = f.readlines()
    for i in range(0, len(lines), 4):
        header = lines[i].rstrip()
        if not header.startswith("@"):
            print("경고: " + filename + "의 " + str(i + 1) + "번째 줄이 @로 시작하지 않습니다: " + header)
        names.append(header[1:])
    return names
```

- `f.readlines()`로 파일의 모든 줄을 리스트로 읽는다.
- `range(0, len(lines), 4)`로 0, 4, 8, … 번째 줄, 즉 read마다 첫 줄(헤더)만 꺼낸다.
- `rstrip()`으로 줄 끝의 줄바꿈 문자를 지운다.
- 헤더가 `@`로 시작하지 않으면 파일이 4줄 단위로 맞지 않는다는 뜻이므로 경고를 출력한다. 경고만 출력하고 프로그램은 계속 진행한다.
- `header[1:]`로 맨 앞의 `@`를 떼고 리스트에 넣는다.
- 결과: read 순서대로 정리된 헤더(이름) 리스트. 리스트 길이가 곧 read 개수이다.

7.2 `read_id(name)` — 비교할 ID 뽑기

```
def read_id(name):
    words = name.split()
    if len(words) == 0:
        return ""
    rid = words[0]
    if rid.endswith("/1") or rid.endswith("/2"):
        rid = rid[:-2]
    return rid
```

- name.split()은 공백을 기준으로 헤더를 나눈다. words[0]이 첫 번째 공백 앞부분이다.
- 예: "SRR1553607.1 1 length=101".split() → ["SRR1553607.1", "1", "length=101"]
- 헤더가 비어 있으면 words[0]에서 오류가 나므로, 그 경우에는 빈 문자열을 돌려준다.
- endswith("/1"), endswith("/2")로 끝부분을 확인하고, 해당하면 rid[:-2]로 마지막 두 글자를 잘라 낸다.
- 예: "HWI:1:1#0/1" → "HWI:1:1#0"

7.3 메인 부분 ① — read 개수 비교

```
names1 = read_names(file1)
names2 = read_names(file2)
count1 = len(names1)
count2 = len(names2)
if count1 == count2:
    count_ok = True
    ...
else:
    count_ok = False
    print("  결과: 불일치 (차이 " + str(abs(count1 - count2)) + "개)")
```

- 두 파일의 헤더 리스트 길이(len)를 비교한다.
- 결과를 count_ok(True/False)에 저장해 두었다가 최종 판정에 쓴다.
- abs()는 절댓값이다. 어느 파일이 더 많아도 차이를 양수로 출력한다.

7.4 메인 부분 ② — 짝이 되는 read 이름 비교

```
n = min(count1, count2)
mismatch = 0
for i in range(n):
    id1 = read_id(names1[i])
    id2 = read_id(names2[i])
    if id1 != id2:
        mismatch = mismatch + 1
        if mismatch <= 5:
            print("  " + str(i + 1) + "번째 read 불일치: " + id1 + " / " + id2)
```

- 개수가 다르면 긴 쪽 리스트의 끝부분에는 짝이 없으므로, min()으로 짧은 쪽 개수만큼만 비교한다. 이렇게 하면 names2[i]에서 IndexError가 나지 않는다.
- 같은 위치 i의 두 헤더에서 ID를 뽑아 !=로 비교하고, 다르면 mismatch를 1 늘린다.
- 다른 쌍이 많을 때 화면이 넘치지 않도록 처음 5개만 자세히 출력한다. 번호는 사람이 읽기 쉽도록 i + 1(1부터 시작)로 출력한다.
- 반복이 끝나면 비교한 쌍 수(n), 일치한 쌍 수(n - mismatch), 다른 쌍 수(mismatch)를 출력한다.
- if n > 0: 안에서 첫 번째 쌍의 실제 헤더와 뽑은 ID를 예시로 출력한다. 빈 파일이면 names1[0]이 없으므로 이 조건으로 오류를 막는다.

7.5 메인 부분 ③ — 최종 판정

```
if count_ok and mismatch == 0:
    print("  두 파일은 짝이 맞는 paired-end FASTQ입니다.")
else:
    print("  두 파일의 짝이 맞지 않습니다.")
```

- read 개수가 같고(count_ok) 이름이 다른 쌍이 하나도 없을 때(mismatch == 0)만 짝이 맞다고 판정한다.

8. 테스트 결과

8.1 실제 데이터

테스트	입력	결과
실제 paired-end 파일	SRR1553607_1.fastq, SRR1553607_2.fastq	read 개수 203,445 / 203,445 일치, 이름이 다른 쌍 0 → 짝이 맞음

8.2 짝이 맞지 않는 경우와 다른 헤더 형식

실제 데이터는 짝이 맞으므로, 틀린 경우도 잡아내는지 작은 테스트 파일을 만들어 확인했다. 테스트 파일은 저장소 밖의 임시 폴더에 만들었다.

테스트	만든 방법	결과
read 개수가 다름	_1에서 read 3개, _2에서 read 2개를 잘라 냄	3 / 2, "불일치 (차이 1개)", 이름이 다른 쌍 0 → 짝이 맞지 않음
이름 하나가 다름	_2의 3번째 read 이름을 SRR1553607.99로 바꿈	"3번째 read 불일치: SRR1553607.3 / SRR1553607.99", 이름이 다른 쌍 1 → 짝이 맞지 않음
예전 일루미나 형식	@HWI:1:1#0/1, @HWI:1:1#0/2	비교에 쓴 ID: HWI:1:1#0 / HWI:1:1#0 → 짝이 맞음
새 일루미나 형식	@M001:1:2 1:N:0:1, @M001:1:2 2:N:0:1	비교에 쓴 ID: M001:1:2 / M001:1:2 → 짝이 맞음
인자를 하나만 줌	python3 src/check_paired.py data/SRR1553607_1.fastq	IndexError: list index out of range

"이름 하나가 다름" 테스트의 실제 출력 일부는 다음과 같다.

```
[2] 짝이 되는 read 이름 확인
  3번째 read 불일치: SRR1553607.3 / SRR1553607.99
  비교한 read 쌍: 3
  이름이 일치한 쌍: 2
  이름이 다른 쌍: 1

[최종 결과]
  두 파일의 짝이 맞지 않습니다.
```

9. 한계

- 파일이 정확히 4줄 단위라고 가정한다. 줄 수가 4의 배수가 아니거나 서열이 여러 줄에 걸친 FASTQ는 올바르게 읽지 못한다 (헤더가 `@`로 시작하지 않으면 경고는 출력한다).
- read 이름만 비교한다. 서열 길이와 품질 문자열 길이가 같은지는 확인하지 않는다.
- 같은 위치끼리만 비교하므로, read가 하나 빠지면 그 뒤의 모든 쌍이 불일치로 나온다. 어디서부터 어긋났는지는 첫 번째 불일치 위치로 알 수 있다.
- 압축 파일(`.fastq.gz`)은 바로 읽지 못한다.

10. 사용한 기본 기능 정리

기능	쓰인 곳
sys.argv	명령줄에서 파일 이름 두 개 받기
open(), readlines()	파일의 모든 줄 읽기
range(0, len(lines), 4)	4줄마다 헤더 줄만 꺼내기
rstrip(), startswith(), endswith()	줄바꿈 제거, 헤더 첫 글자 확인, /1 /2 확인
split()	공백 기준으로 헤더 나누기
문자열 슬라이싱 [1:], [:-2]	@ 떼기, /1 /2 떼기
len(), min(), abs()	read 개수, 비교할 쌍 수, 개수 차이
if / for	조건 판정과 반복 비교